

Your AI Agent Is About to Spend Money. Who's Going to Trust It?

Introducing AVP-Micro: a portable trust and authorization layer for autonomous-agent payments.

Picture an AI agent halfway through a task. To finish, it needs to call a paid model, pull a slice of fresh data, and rent a few seconds of compute. Three tiny purchases, each a fraction of a cent, all of them happening in the next 200 milliseconds — with no human anywhere near the keyboard.

The money is the easy part. We already have rails that move value fast and cheap: Lightning, Interledger, stablecoins, the new HTTP-402 flows like x402 and Stripe's Machine Payments Protocol. The hard part is everything *around* the money. When that agent arrives at a checkout, nobody on the other side can answer three basic questions:

- **Who is paying?**
- **Under whose authority, and within what limits?**
- **Did the money actually move, and can we prove it later?**

Today we paper over those questions with API keys, OAuth tokens, and platform accounts — instruments designed for humans logging into apps, not for software spending real money on its own. They're coarse ("this key can do anything the account can"), opaque (you can't read a spending limit off a bearer token), and locked to one platform. That's a shaky foundation for an economy where agents transact thousands of times an hour, across organizations they've never integrated with before.

AVP-Micro is a specification built to fix exactly that gap.

The idea in one sentence

AVP-Micro adds a **portable, verifiable trust and authorization layer on top of any payment rail** — so the rail handles *moving the money* while AVP-Micro handles *identity, authority, and proof*.

The design leans on two mature W3C standards:

- **Decentralized Identifiers (DIDs)** give every participant — the agent, the merchant, the wallet — a cryptographic identity it controls, independent of any single platform. This answers *who*.
- **Verifiable Credentials (VCs)** let a principal hand an agent a signed, machine-checkable mandate: *this agent may spend up to this much, with these merchants, until this date*. This answers *under what authority*.

Everything is signed, and every step leaves behind a self-describing record anyone can verify. That answers *did it happen*.

Crucially, this layer is **rail-agnostic**. The same agent, carrying the same credential, can settle over Interledger for one purchase and a card network for the next. AVP-Micro standardizes the trust; it never tries to replace the plumbing that moves value.

Two specifications, cleanly separated

AVP-Micro is published as two peer specifications, so each layer can be adopted — and reasoned about — on its own.

Delegated Spending Authority (DSA) is the identity and trust foundation. It defines the **SpendingAuthorizationCredential** — the signed mandate a principal issues to its agent — along with merchant credentials, payment-capability credentials, and the rules a wallet uses to decide which credential issuers it trusts. In plain terms, a spending authorization says something like:

*Agent **maintenance-bot-01** may spend up to **\$0.05 per transaction, \$5.00 per day, in USD**, only with the **HVAC control service** — and this mandate expires next Friday.*

That's it: a constraint a human can read and a machine can enforce, signed by the person or organization who owns the money.

AVP-Micro Payments builds on DSA to define the actual payment messages: the **quote** a merchant offers, the **authorization** the agent sends back (carrying its credential), the **execution** the wallet performs on the chosen rail, and the **receipt** the merchant signs once the service is delivered. It also defines a **streaming mode** for metered, continuous usage — more on that below.

The split matters strategically. An organization can stand up verifiable agent *identity and spending limits* (DSA) before it commits to any particular payment flow, and merchants can verify agent authority without caring which rail settles the bill.

How a payment actually flows

A one-off purchase moves through a short, predictable sequence:

1. **Discover & quote.** The agent asks a merchant what a specific request will cost. The merchant returns a signed quote — an amount, a currency, a settlement target, and a short expiry — bound to a hash of the exact request so it can't be quietly swapped later.
2. **Authorize.** The agent checks the quote against its own spending limits, then sends back a signed authorization that *embeds its spending credential*. This is the moment the agent proves both intent ("I accept this quote") and authority ("and here's my signed mandate to do so").
3. **Settle.** The wallet verifies the signatures and the policy — amount within limit, merchant allowed, credential valid and unrevoked — and only then moves the money on the chosen rail.
4. **Receipt.** The merchant delivers the service and signs a receipt that ties the payment to what was actually delivered.

When the agent's workload is continuous rather than one-shot — streaming sensor data, or paying per token of model output — a round-trip per micro-charge is wasteful. AVP-Micro's **session mode** handles this: the agent opens a usage session against a pre-approved budget, the merchant reports incremental **usage accruals** as work is consumed, and settlement happens incrementally or in one consolidated payment at the end. The session can even be extended and re-authorized mid-stream without tearing everything down.

Why you can trust the result

Trust here isn't a promise; it's a property of the artifacts. A few design choices make it hold up:

- **Everything is signed.** Messages and credentials use W3C Data Integrity proofs with the `eddsa-jcs-2022` cryptosuite over `did:key` identities — a concrete, interoperable, mandatory-to-implement baseline, not a hand-wave.
- **Everything is bound.** A nonce and a short expiry stop replay attacks. A request hash ties each authorization to one exact purchase, so a stale or tampered quote simply fails verification.
- **Everything is auditable.** Because each quote, authorization, execution, and receipt is a self-contained, cryptographically signed object, the full chain can be replayed and verified after the fact — by the principal, an auditor, or a regulator — without trusting any intermediary's database.

The spec ships with signed test vectors for both the one-off and streaming flows, and a validation harness that checks proofs, credential bindings, and policy end to end. The trust story is testable, not aspirational.

Where this shows up in practice

A pay-per-call API marketplace. An agent shops across model and data providers it has never integrated with, paying each per request. The same DID and credentials work everywhere; no provider needs a bespoke onboarding for the agent, and the principal gets a verifiable record of every cent.

Metered, streaming usage. An agent consumes a real-time sensor feed or a token-by-token model stream. It opens one session against a capped budget, usage accrues, and a single consolidated receipt closes it out — micropayment economics without micropayment overhead, and a hard ceiling the wallet enforces so the agent can't overspend.

Cross-domain B2B procurement. A procurement agent buys from several suppliers across organizational boundaries, each settling over whatever rail it prefers. Months later, finance presents the signed receipts and authorizations to an auditor who verifies the entire chain — who authorized what, within which limits, and what was delivered — from the artifacts alone.

The thread running through all three: **portable authority and built-in proof.** The agent carries its identity and its limits with it, and every transaction explains itself.

Where it fits, and what's next

AVP-Micro deliberately sits *alongside* the work already happening in agent payments rather than competing with it. It composes with W3C's DID and Verifiable Credentials data models, IETF message-signing work, and the HTTP-402 payment flows emerging from Interledger, x402, and Stripe's MPP. Those efforts are largely about moving value and negotiating a charge; AVP-Micro contributes the missing piece — *portable identity and verifiable, fine-grained authority* — that lets any of them be trusted across organizational lines.

The specification is a submission candidate today: two peer documents, JSON-LD contexts, JSON Schemas, SHACL shapes, an ontology, and signed test vectors, all validated by a reproducible harness.

If you're thinking about how autonomous agents in your product will pay for what they consume — and how you'll prove, later, that they stayed within bounds — AVP-Micro is worth a close read. The money was never the hard part. Trust is. This is a specification for getting it right.

AVP-Micro is published as two peer specifications — Delegated Spending Authority and AVP-Micro Payments — with contexts, schemas, shapes, and signed test vectors. Start with the spec README to explore the full normative documents and run the validation harness yourself.